

# EDUGENIE

## AI-Powered Learning Assistant

Project Documentation & Representative Source Code

**Technology:** Python • FastAPI • Google Gemini • HTML • CSS • JavaScript • Pydantic

# 1. Introduction

EduGenie is an AI-powered educational learning assistant designed to help students learn academic concepts using Generative Artificial Intelligence.

The application provides multiple learning features:

- Question and Answer
- Topic Explanation
- Quiz Generation
- Text Summarization
- Personalized Learning Path

The application uses a FastAPI backend and a web-based frontend. Google Gemini is used as the main cloud-based Generative AI service.

# 2. Objectives

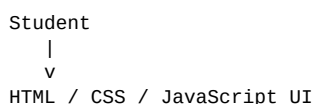
1. Provide students with quick answers to academic questions.
2. Explain difficult concepts in simple language.
3. Automatically generate multiple-choice questions.
4. Summarize lengthy educational content.
5. Recommend a structured learning path.
6. Provide an easy-to-use web interface.
7. Demonstrate the practical use of Generative AI in education.

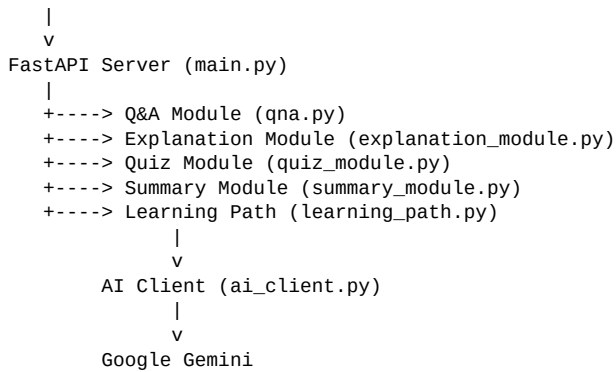
# 3. Technology Stack

| Component                 | Technology            |
|---------------------------|-----------------------|
| Frontend                  | HTML, CSS, JavaScript |
| Backend                   | Python                |
| Web Framework             | FastAPI               |
| AI/LLM                    | Google Gemini         |
| Optional Local Model      | LaMini-Flan-T5        |
| API Format                | JSON                  |
| Template Engine           | Jinja2                |
| Validation                | Pydantic              |
| Server                    | Uvicorn               |
| Environment Configuration | python-dotenv         |

# 4. Project Architecture

## Architecture





The architecture separates the web layer, educational modules, AI client, validation, configuration, and utility functions. This makes the application easier to maintain and extend.

## 5. Project Folder Structure

### Folder structure

```

EduGenie/
├── main.py
├── config.py
├── ai_client.py
├── schemas.py
├── utils.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── requirements-local.txt
├── .env
├── .gitignore
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── app.js
├── tests/
│   ├── __init__.py
│   └── test_api.py

```

## 6. Backend Documentation

### 6.1 main.py

**Purpose:** Main entry point of the EduGenie FastAPI application.

- Creates the FastAPI application.
- Configures CORS.
- Serves static files.
- Loads the HTML template.
- Defines API endpoints.
- Connects frontend requests to AI modules.

**Main endpoints:** GET /, GET /health, POST /qa, POST /explain, POST /quiz, POST /summarize, POST /learn/recommendations.

### Representative main.py

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse

from qna import answer_question
from explanation_module import explain_topic

```

```

from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import recommend_learning_path
from schemas import TextRequest

app = FastAPI(title="EduGenie", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/", response_class=HTMLResponse)
def home():
    with open("templates/index.html", encoding="utf-8") as f:
        return f.read()

@app.get("/health")
def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/qa")
def qa(req: TextRequest):
    return answer_question(req.text)

@app.post("/explain")
def explain(req: TextRequest):
    return explain_topic(req.text)

@app.post("/quiz")
def quiz(req: TextRequest):
    return generate_quiz(req.text)

@app.post("/summarize")
def summarize(req: TextRequest):
    return summarize_text(req.text)

@app.post("/learn/recommendations")
def learn(req: TextRequest):
    return recommend_learning_path(req.text)

```

## 6.2 config.py

Reads environment variables from .env and keeps configuration separate from application logic.

### Representative config.py

```

import os
from dotenv import load_dotenv

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-flash")
USE_LOCAL_EXPLANATION = os.getenv(
    "USE_LOCAL_EXPLANATION", "false"
).lower() == "true"
LOCAL_EXPLANATION_MODEL = os.getenv(
    "LOCAL_EXPLANATION_MODEL",
    "MBZUAI/LaMini-Flan-T5-783M"
)
HOST = os.getenv("HOST", "127.0.0.1")
PORT = int(os.getenv("PORT", "8000"))

```

## 6.3 ai\_client.py

Provides a common interface for communicating with Gemini. A shared function avoids creating separate AI clients in every module.

### Representative ai\_client.py

```

from google import genai
from config import GEMINI_API_KEY, GEMINI_MODEL

client = genai.Client(api_key=GEMINI_API_KEY)

def generate_text(

```

```

    prompt: str,
    temperature: float = 0.3,
    max_output_tokens: int = 1024,
    json_mode: bool = False
):
    config = {
        "temperature": temperature,
        "max_output_tokens": max_output_tokens,
    }

    if json_mode:
        config["response_mime_type"] = "application/json"

    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=prompt,
        config=config,
    )
    return response.text

```

Note: Gemini SDK syntax can vary by installed SDK version. Treat this as representative project code and verify it against the version in requirements.txt.

## 6.4 schemas.py

Pydantic models define the expected request and response structure.

### Representative schemas.py

```

from pydantic import BaseModel
from typing import List

class TextRequest(BaseModel):
    text: str

class QuizQuestion(BaseModel):
    question: str
    options: List[str]
    answer: str
    explanation: str

class QuizResponse(BaseModel):
    questions: List[QuizQuestion]

```

## 6.5 utils.py

Utility functions clean AI-generated JSON and convert it into Python objects.

### Representative utils.py

```

import json

def clean_json_block(text: str) -> str:
    text = text.strip()
    if text.startswith("```"):
        text = text.replace("```json", "", 1)
        text = text.replace("```", "")
    return text.strip()

def parse_json_response(text: str):
    return json.loads(clean_json_block(text))

```

## 7. Q&A; Module — qna.py

The Q&A; module answers student questions using a concise educational prompt.

### Representative qna.py

```

from ai_client import generate_text

def answer_question(question: str):
    prompt = f"""
    You are an educational assistant.
    Answer the student's question using simple language.
    Explain difficult terms and use bullet points when useful.

    Question:
    {question}
    """

```

```

"""
    answer = generate_text(prompt, temperature=0.3)
    return {"question": question, "answer": answer}

```

## 8. Explanation Module — explanation\_module.py

The explanation module presents topics using a beginner-friendly structure: Simple Definition, How It Works, Easy Example, and Important Points.

### Representative explanation\_module.py

```

from ai_client import generate_text

def explain_topic(topic: str):
    prompt = f"""
    Explain the academic topic below for a beginner.

    Topic: {topic}

    Use this structure:
    1. Simple Definition
    2. How It Works
    3. Easy Example
    4. Important Points
    """
    return {
        "topic": topic,
        "explanation": generate_text(prompt, temperature=0.4)
    }

```

The optional local mode can use LaMini-Flan-T5 when USE\_LOCAL\_EXPLANATION=true. Gemini is the default mode.

## 9. Quiz Module — quiz\_module.py

The quiz module requests three multiple-choice questions, four options per question, one correct answer, and an explanation. The response is validated before being returned.

### Representative quiz\_module.py

```

from ai_client import generate_text
from utils import parse_json_response

def generate_quiz(topic: str):
    prompt = f"""
    Create exactly 3 multiple-choice questions about: {topic}

    Return ONLY valid JSON:
    {{
      "questions": [
        {{
          "question": "...",
          "options": ["A", "B", "C", "D"],
          "answer": "...",
          "explanation": "..."
        }}
      ]
    }}
    """
    data = parse_json_response(
        generate_text(prompt, temperature=0.2, json_mode=True)
    )

    questions = data.get("questions", [])
    if len(questions) != 3:
        raise ValueError("Quiz must contain exactly 3 questions")

    for q in questions:
        if len(q.get("options", [])) != 4:
            raise ValueError("Each question must contain 4 options")
        if q["answer"] not in q["options"]:
            raise ValueError("Answer must match one of the options")

    return data

```

## 10. Summary Module — summary\_module.py

Converts lengthy educational content into concise revision material while preserving important information.

### Representative summary\_module.py

```
from ai_client import generate_text

def summarize_text(text: str):
    prompt = f"""
    Summarize the educational content below for revision.
    Preserve important information, remove repetition,
    use simple English, and do not add unrelated facts.

    Content:
    {text}
    """
    summary = generate_text(prompt, temperature=0.2)
    return {"summary": summary}
```

## 11. Learning Path — learning\_path.py

Creates a structured plan progressing from Beginner to Intermediate, Advanced, Mini Project, Learning Resources, and a Four-Week Plan.

### Representative learning\_path.py

```
from ai_client import generate_text
from utils import parse_json_response

def recommend_learning_path(topic: str):
    prompt = f"""
    Create a four-week learning path for: {topic}

    Include:
    - Beginner
    - Intermediate
    - Advanced
    - Mini Project
    - Learning Resources
    - Four-Week Plan

    Return the result as valid JSON.
    """
    result = generate_text(prompt, temperature=0.3, json_mode=True)
    return parse_json_response(result)
```

## 12. Frontend Documentation

### 12.1 index.html

The main interface contains the EduGenie title, task selection, text input, Generate and Clear buttons, result area, and API documentation link.

### Representative index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <header><h1>EduGenie</h1></header>

  <main>
    <select id="task">
      <option value="qa">Question & Answer</option>
      <option value="explain">Explain Topic</option>
      <option value="quiz">Generate Quiz</option>
      <option value="summarize">Summarize</option>
      <option value="learn">Learning Path</option>
    </select>

    <textarea id="input" placeholder="Enter your topic or question"></textarea>
```

```

    <button id="generate">Generate</button>
    <button id="clear">Clear</button>

    <section id="result"></section>
  </main>

  <script src="/static/app.js"></script>
</body>
</html>

```

## 12.2 style.css

Controls headers, cards, buttons, text areas, dropdowns, quiz options, and responsive mobile layout.

### Representative style.css

```

body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f5f7fb;
}

main {
  max-width: 900px;
  margin: 30px auto;
  padding: 20px;
}

textarea, select, button {
  width: 100%;
  margin: 8px 0;
  padding: 12px;
  box-sizing: border-box;
}

textarea {
  min-height: 180px;
}

button {
  cursor: pointer;
}

@media (max-width: 600px) {
  main { margin: 10px; padding: 10px; }
}

```

## 12.3 app.js

Reads the selected task, sends the appropriate API request, receives JSON, displays results, handles quiz interactions, and reports errors.

### Representative app.js

```

const endpoints = {
  qa: "/qa",
  explain: "/explain",
  quiz: "/quiz",
  summarize: "/summarize",
  learn: "/learn/recommendations"
};

async function generate() {
  const task = document.getElementById("task").value;
  const text = document.getElementById("input").value.trim();
  const result = document.getElementById("result");

  if (!text) {
    result.textContent = "Please enter a question or topic.";
    return;
  }

  result.textContent = "Generating...";

  try {
    const response = await fetch(endpoints[task], {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({text})
    });
  }
}

```

```

    if (!response.ok) {
      throw new Error("API request failed");
    }

    const data = await response.json();
    result.textContent = JSON.stringify(data, null, 2);
  } catch (error) {
    result.textContent = error.message;
  }
}

document.getElementById("generate").addEventListener("click", generate);

document.getElementById("clear").addEventListener("click", () => {
  document.getElementById("input").value = "";
  document.getElementById("result").textContent = "";
});

```

## 13. requirements.txt

### Dependencies

```

fastapi
uvicorn
jinja2
python-dotenv
pydantic
google-generativeai
pytest

```

Install with: `pip install -r requirements.txt`

## 14. Environment Configuration — .env

### .env example

```

GEMINI_API_KEY=YOUR_API_KEY
GEMINI_MODEL=gemini-flash
USE_LOCAL_EXPLANATION=false
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M
HOST=127.0.0.1
PORT=8000

```

Never commit a real API key to source control. Add `.env` to `.gitignore`.

## 15. Testing

The project contains basic API tests for the home page, health endpoint, and request validation.

### Representative tests/test\_api.py

```

from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_home_page():
    response = client.get("/")
    assert response.status_code == 200

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_empty_input():
    response = client.post("/qa", json={"text": ""})
    assert response.status_code in (200, 422)

```

### Run tests

```
python -m pytest
```

## 16. Installation Procedure

Step 1: Open the EduGenie folder in VS Code.

Step 2: Create a virtual environment: `python -m venv .venv`

Step 3: Windows PowerShell: `.venv\Scripts\Activate.ps1`

Step 4: Install dependencies: `pip install -r requirements.txt`

Step 5: Configure the `.env` file with your Gemini API key and model.

Step 6: Run the server: `python -m uvicorn main:app --reload`

Step 7: Open `http://127.0.0.1:8000` in a browser.

## 17. API Documentation

FastAPI automatically provides interactive API documentation at `/docs`.

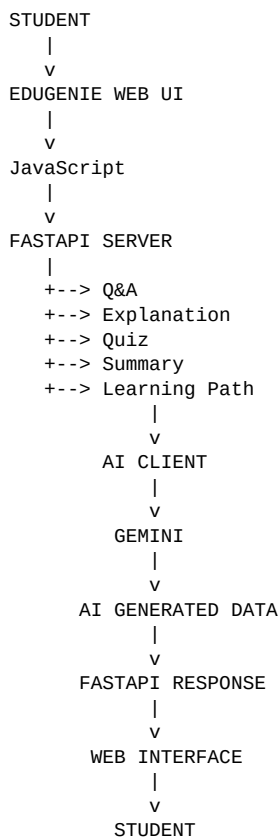
### API documentation URL

`http://127.0.0.1:8000/docs`

The interface can be used to test `/qa`, `/explain`, `/quiz`, `/summarize`, and `/learn/recommendations` without using the frontend.

## 18. Complete Application Workflow

### End-to-end workflow



## 19. Advantages

- AI-based academic assistance.
- Simple explanations.
- Automatic quiz generation.
- Quick revision through summaries.

- Personalized learning paths.
- Web-based interface.
- Modular backend architecture.
- REST API support.
- Interactive API documentation.
- Optional local AI model support.

## 20. Future Enhancements

1. Student login and authentication.
2. Database integration.
3. Student progress tracking.
4. Quiz scores and history.
5. PDF/document upload.
6. Voice-based questions.
7. Multi-language support.
8. Personalized recommendations based on previous performance.
9. Teacher dashboard.
10. Mobile application.

## 21. Conclusion

EduGenie demonstrates how Generative AI can be integrated into an educational web application. The system combines a frontend, FastAPI backend, AI/LLM integration, educational modules, and REST APIs to create an interactive learning assistant.

The application can answer questions, explain topics, generate quizzes, summarize content, and create learning paths. Its modular structure makes the system easier to understand, maintain, test, and extend.

## 22. Quick Reference

| Feature       | Endpoint                    | Module                |
|---------------|-----------------------------|-----------------------|
| Q&A           | POST /qa                    | qna.py                |
| Explanation   | POST /explain               | explanation_module.py |
| Quiz          | POST /quiz                  | quiz_module.py        |
| Summary       | POST /summarize             | summary_module.py     |
| Learning Path | POST /learn/recommendations | learning_path.py      |
| Health        | GET /health                 | main.py               |

**Security reminder:** Keep GEMINI\_API\_KEY private and do not publish real credentials in the PDF, GitHub repository, screenshots, or source code.